Código de tres direcciones de Compiscript

Este documento describe el TAC escalar y de flujo de control, de funciones léxicas, de objetos y colecciones, de valores desconocidos protegidos y de try/catch para la Entrega 2. El informe registra la matriz final del PDF, las brechas históricas resueltas y los controles humanos pendientes. Las funciones usan llamadas y retornos explícitos y registros de activación deterministas en celdas abstractas, incluida la recursión directa y mutua, los closures retenidos y los alias de función inferidos. Los objetos incluyen campos heredados, construcción, métodos virtuales, receptores retenidos y cadenas. Las listas incluyen referencias, literales, lecturas y escrituras indexadas y foreach. El JSON de compilación incluye símbolos extendidos y disposiciones de almacenamiento; el IDE usa el mismo núcleo y el texto TAC generado en el servidor. La API de biblioteca independiente `compiscript.Compile(source []byte)` genera TAC tipado y ubicado en la fuente para todos los tipos de construcción válidos del AST, con operaciones tipadas y protegidas, flujo de control, llamadas/retornos y manejadores. Los contratos de análisis existentes se mantienen sin cambios, salvo la corrección del mapeo de encabezados `for` opcionales y la recuperación ante cuerpo ausente descritas más abajo. El comando independiente `compiscript-tac` compila este lenguaje sin cambiar la interfaz ni la política de códigos de salida del comando de análisis existente.

Ruta rápida del comando

Ejecute el ejemplo numérico incluido (calcula el TAC de `x = 3.5`, sin ejecutar el programa fuente):

```
go run ./cmd/compiscript-tac testdata/compiscript/tac/numeric.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/numeric.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/conditions.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/conditions.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/loops.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/loops.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/switch.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/switch.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/functions.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/functions.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/recursion.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/recursion.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/closures.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/closures.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/collections.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/collections.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/objects.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/objects.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/acceptance.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/acceptance.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/values-effects.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/values-effects.cps
go run ./cmd/compiscript-tac testdata/compiscript/tac/dynamic-exceptions.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/dynamic-exceptions.cps
go run ./cmd/compiscript-tac -h
```

El formato por defecto escribe solo texto TAC en stdout. Una compilación rechazada escribe diagnósticos ubicados en stderr, nada de TAC en stdout, y termina con código 1. El formato JSON escribe el `CompilationReport` completo en stdout, incluido `tac: null` en caso de rechazo; su código de salida sigue siendo 1 cuando existen diagnósticos. El éxito termina con 0, incluso con una fuente vacía. Los argumentos o formatos inválidos terminan con 2; los fallos de archivo/UTF-8/escritura terminan con 1; la ayuda termina con 0 e imprime el uso en stderr. Las banderas preceden a la ruta de la fuente. `go run` puede envolver el código de salida distinto de cero de un programa; construya el binario cuando una automatización necesite el estado exacto del comando. Las instantáneas de ámbitos siguen siendo metadatos semánticos; las disposiciones de marco (frame) de los procedimientos son metadatos separados de generación de código, descritos más abajo.

Contrato de símbolos de compilación (T6)

Las respuestas exitosas de `Compile` y de la CLI con `-format json` agregan un objeto opcional de nivel superior `codegen`. Una compilación rechazada lo omite por completo (`tac` sigue siendo null). Los ámbitos heredados de Analyze y el TAC JSON/texto soportado previamente no cambian. Los consumidores pueden ignorar esta adición; no infiera el almacenamiento a partir de los ámbitos semánticos mismos.

Campo	Tipo JSON	Significado
`codegen.unit`	string	Siempre `"abstract-cell"`; cada escalar o referencia ocupa una celda
`codegen.symbols`	array	Todos los símbolos semánticos de la fuente, ordenados por la identidad estable `s<scopeId>.<name>`
`symbols[].id`, `scopeId`, `symbol`	string, integer, object	Identidad del enlace, ID del ámbito original y Symbol semántico sin cambios (nombre/tipo de símbolo/tipo/mutabilidad/captura/span)
`symbols[].locations`	array	Cada ubicación real de preparación (staging), retenida o de instancia; vacío para símbolos sin almacenamiento o no emitidos
`locations[]`	object	`space` string, `offset` integer, `size` integer, `role` string
`symbols[].labels`	string array	Etiquetas reales de procedimientos, o ID de la disposición de clase más la etiqueta del inicializador; no hay celda ficticia para una etiqueta
`codegen.spaces`	array	Strings `id`/`kind`, `scopeId` entero opcional, `parent` string opcional, `cells` entero y arreglo `slots`
`spaces[].slots`	array	Forma existente de FrameSlot: `offset`/`size` enteros, `role` string, Operand `operand` opcional, booleano `dynamic` opcional
`codegen.labels`	array	Strings `name`, `kind`, `owner` y `span` de la fuente; tipos: `procedure`, `control`, `classLayout`, `method`

Para `let x=1+2;`, la entrada del símbolo es la siguiente (el objeto `symbol` real también contiene el tipo original, las banderas y el span de la fuente):


```
{
  "id": "s1.x",
  "scopeId": 1,
  "symbol": {"name":"x","kind":"variable","type":{"kind":"integer","name":"integer","element":null,"par
  "locations": [{"space":"globals","offset":0,"size":1,"role":"binding"}],
  "labels": []
}
```


El espacio `globals` correspondiente tiene un slot de enlace; `main` tiene un slot temporal dinámico para `t0`. Todos los arreglos están presentes incluso cuando están vacíos. Los desplazamientos (offsets) comienzan en cero **dentro del espacio nombrado** y nunca son direcciones físicas:

- `globals`: símbolos emitidos que no pertenecen a un marco ni a un entorno, ordenados por identidad. Incluye los enlaces del bloque principal, no solo las declaraciones globales de la fuente.
- `main`: celdas temporales realmente usadas por las instrucciones emitidas de main, en el orden de numeración de temporales del emisor. Las celdas reutilizadas no tienen un tipo permanente (`dynamic: true`).
- `frame:<procedure-label>`: copia exactamente el `Procedure.Frame` existente, incluida la continuación (dirección de retorno abstracta), los enlaces de control y de acceso global, el resultado opcional, el parámetro receptor, los parámetros ordinarios, los locales y los temporales.
- `environment:s<scopeId>`: enlaces en el orden existente de EnvironmentLayout; `parent` registra el enlace léxico existente entre disposiciones, no una celda adicional fabricada.
- `instance:<class-id>`: desplazamientos de campo existentes, incluidos los campos heredados. `parent` identifica la disposición base; las etiquetas de métodos asocian las entradas de despacho efectivas de cada clase con los procedimientos emitidos, incluidos los constructores heredados.

Un parámetro o local puede tener tanto un slot de preparación en el marco como un enlace retenido en un entorno. Un campo heredado aparece en cada disposición de instancia relevante. El informe lista todas esas ubicaciones; no inventa una única dirección física canónica. Los IDs de ámbito identifican disposiciones, no invocaciones vivas ni instancias en el heap. Las cadenas, listas, objetos y closures ocupan celdas de referencia; el tamaño de su contenido y su dirección viva son desconocidos, y no se afirma ninguna ABI física. Los campos

existentes `Procedure.lexicalScope/captures` y las disposiciones padre de los entornos describen el acceso léxico; el slot existente `globalAccessLink` del marco no se renombra ni se reutiliza para otro fin.

Decisión KISS: derivar este inventario de los operandos emitidos y de las disposiciones existentes de marcos, entornos y clases, y luego unirlos por el ID semántico del enlace. No se introduce un segundo análisis sintáctico, otro análisis ni otro modelo de enlace. Las constantes obtienen almacenamiento solo cuando se emiten; las etiquetas de procedimientos, de control y de disposición, así como los símbolos de la fuente no emitidos, no reciben desplazamientos ni tamaños inventados. Las etiquetas de control incluyen su espacio propietario `main`/marco porque sus nombres pueden repetirse entre procedimientos. Este es un informe estático, no un intérprete en tiempo de ejecución. La arquitectura describe sus consumidores en la CLI y el IDE.

Representación

Una instrucción tiene un código de operación (opcode), un destino, un operando izquierdo, un operando derecho opcional y el span de la fuente del AST que la originó. El campo opcional `label` nombra un destino de flujo de control; se omite en el JSON numérico, lo que preserva el orden de campos y la salida existentes. Los operandos distinguen literales, símbolos y temporales virtuales, y conservan el tipo semántico `integer`, `float` o `boolean`. La representación escalar se extiende a valores de cadena, referencia y null, y a valores protegidos de tipo `error` descritos más abajo. El texto representa un operando como `value:type`; el JSON estructurado conserva su clase. Esta representación no es SSA: una asignación puede actualizar un símbolo o reutilizar un temporal. Los spans ubican operaciones de la fuente, no identidades únicas de expresiones.

Instrucción	Texto	Significado
Copia	`s1.x:integer = 1:integer`	Asigna un valor a un símbolo o temporal
Suma/resta/multiplicación	`t0:integer = 2:integer * 3:integer`	Operación numérica
División	`t0:float = 6.0:float / 2.0:float`	División de punto flotante
Residuo	`t0:float = 7.5:float % 2.0:float`	Residuo truncado; un divisor cero produce un fallo
Impresión	`print s1.x:integer`	Lee `left` una vez y agrega una línea de salida; sin destino
Declaración	`declare s1.x:integer`	Activa/reserva un enlace con una celda indefinida; no hay escritura por defecto
Negación	`t0:integer = neg s1.x:integer`	Menos unario numérico
Entero a flotante	`t0:float = i2f s1.x:integer`	Ampliación numérico explícito
Comparación	`t0:boolean = 1:integer < 2:integer`	`==`, `!=`, `<`, `<=`, `>`, `>=`; también se soporta igualdad/desigualdad booleana
No lógico	`t0:boolean = ! s1.ready:boolean`	Negación booleana
Etiqueta	`L0:`	Destino de flujo de control
Salto	`goto L0`	Arista incondicional
Bifurcación si verdadero	`if t0:boolean goto L0`	Salta cuando el operando izquierdo booleano es verdadero; en otro caso continúa
Llamada	`t0:integer = call s1.f(2:integer)`	Evalúa los valores de los argumentos en el marco del llamador, invoca el procedimiento nombrado con almacenamiento nuevo, reanuda y escribe su resultado tipado
Llamada con resultado null	`call s1.update()`	Invoca sin una celda de resultado del llamado; un uso como expresión produce null después de los efectos
Retorno	`return t0:integer` o `return`	Transfiere un resultado verificado, o ningún valor, a la continuación guardada del llamador

Las instrucciones de etiqueta y salto usan `label`; la bifurcación si verdadero también usa `left`. Su operando heredado `target`, no utilizado, tiene valor cero; el `left` de etiqueta/salto también tiene valor cero, y `right` se omite. Estos campos no son lecturas ni escrituras de valores. Las etiquetas comienzan en `L0` y se asignan de forma determinista por cada cinta de instrucciones (main o procedimiento). El texto y el JSON de las instrucciones numéricas existentes no cambian.

Cada instrucción lee todos sus operandos fuente antes de escribir su destino. En consecuencia, un destino puede reutilizar el temporal del último operando consumido. Un entero denota un entero exacto abstracto; aquí no se seleccionan un ancho ni una ABI específicos de la máquina destino. Los flotantes denotan valores IEEE-754 binary64; el ampliación de enteros puede redondear. Las operaciones mixtas ensanchan explícitamente los enteros. La división ensancha los operandos enteros y produce un flotante, en concordancia con el analizador semántico. No se realiza plegado de constantes; la división entre cero es un fallo en tiempo de ejecución para un backend futuro, no un permiso para omitir instrucciones. No hay un ejecutor de TAC de producto en esta unidad.

Enlace y tiempos de vida

El compilador de fuente asigna nombres de símbolo `s<scope-id>.<source-name>` a partir del inventario semántico de ámbitos. Son identidades lógicas, no desplazamientos en bytes, y son deterministas para la misma fuente, pero no estables ante ediciones de la fuente. Las declaraciones de variables se vuelven visibles solo después de la traducción (lowering) de su inicializador. Esto preserva el enlace a la variable externa en un inicializador que la sombrea. Las identidades y firmas de las funciones con nombre se predeclaran para permitir llamadas hacia adelante; los enlaces de variables no. El hoisting se aplica a cada secuencia de sentencias, incluidos los cuerpos anidados y los casos de switch, sin inventar ámbitos semánticos adicionales. Los ámbitos de función, los registros de activación y las capturas retenidas se describen más abajo.

Los temporales comienzan en `t0`. La asignación elige el slot numérico disponible más pequeño; la liberación ocurre solo cuando se emite la instrucción consumidora final. Cada ocurrencia conserva su propio tipo, de modo que un slot libre puede reutilizarse para un tipo distinto. Los operandos vivos no pueden solaparse; las sentencias repetidas no reservan slots nuevos indefinidamente. `peakTemporaries` registra la demanda máxima de temporales vivos, no el uso de registros físicos, el tamaño de pila ni una afirmación de optimalidad global.

Funciones directas y registros de activación abstractos

Firmas, enlace y validación

Las funciones con nombre soportan contratos tipados numéricos, booleanos, de cadena, de clase y de lista, así como parámetros formales sin tipo protegidos. Los resultados siguen su contrato semántico declarado; el resultado del analizador sigue siendo `null` cuando se omite. Por ejemplo:

```
function increment(value: integer): integer {
    value = value + 1;
    return value;
}
let answer = increment(2);
```

Las llamadas se enlazan a través del entorno léxico activo, nunca mediante una búsqueda en un registro solo por nombre. Los alias de función inferidos con `let/const` y las asignaciones a alias mutables conservan valores de función; un alias que sombrea el nombre de una función es invocable. El sombreado con algo que no es función sigue siendo inválido. La gramática existente usa identificadores como llamados; no se agrega sintaxis de lambdas ni de anotaciones de tipo función. Los parámetros sin tipo usan valores etiquetados protegidos; las funciones externas del AST sin cuerpo y las firmas malformadas se rechazan explícitamente. Una función usa el ámbito semántico de función existente, que abarca toda su declaración; sus parámetros y las declaraciones directas de su cuerpo comparten ese ámbito. Los bloques anidados reales y los ámbitos de `for` conservan sus propios IDs semánticos. Los parámetros son valores mutables locales a la invocación, no alias de símbolos del llamador.

Cada cuerpo se traduce una sola vez en orden de la fuente, incluido el código no llamado e inalcanzable, con un pool de temporales existente independiente y una pila de transferencias nueva. Todas las aristas de llamada directa se recolectan sin traducir recursivamente a los llamados. La recursión propia y mutua usa los mismos marcos locales a la invocación y continuaciones de retorno explícitas que las demás llamadas. Los ciclos no llamados son válidos, pero cada cuerpo pasa igualmente por la validación estática. La compilación no garantiza la terminación en tiempo de ejecución.

Los retornos escalares usan la compatibilidad de tipos existente y el ampliación explícito `i2f`; no hay un valor escalar por defecto sintetizado. Los diagnósticos semánticos de retorno faltante siguen siendo la autoridad. Los procedimientos con resultado null aceptan return sin valor, return null (también null agrupado) y caída al final; la caída al final emite un retorno explícito sin valor. Las llamadas con resultado null usadas como expresión se ejecutan exactamente una vez, sin destino ni celda de resultado en el marco, y luego producen el literal `null`. Una expresión de retorno null se evalúa por sus efectos antes del retorno sin valor. Las anotaciones de resultado omitidas siguen significando null, no tipos de retorno inferidos.

Evaluación e inicialización global

Los operandos y argumentos se evalúan de izquierda a derecha, una sola vez. Antes de evaluar un subárbol posterior con efectos (incluidas las asignaciones), un operando símbolo anterior se copia en un temporal reservado. Esto se aplica a la aritmética, las comparaciones, las listas de argumentos y al selector de un switch antes de pruebas de caso con efectos. Los literales y los temporales ya vivos no necesitan copia. Para `let g=1; ... let x=g+bump();`, donde `bump` incrementa y devuelve `g`, `x` vale 3, no 4. De forma similar, `pair(g,bump())` recibe 1 y 2, no 2 y 2. El ampliación ocurre antes de evaluar los argumentos posteriores. Los operadores lógicos `&&`/`||` conservan el salto por cortocircuito en tiempo de ejecución, mientras ambos operandos se validan estáticamente. Los programas sin llamadas ni efectos de asignación no reciben instrucciones de instantánea adicionales.

Los cuerpos de función conservan la visibilidad global del momento de su declaración y los nombres bloqueados por switch, no el inventario final de variables. Las lecturas y escrituras globales se permiten cuando están inicializadas; la ausencia de una bandera semántica `Captured` en las globales no proporciona aristas de uso a enlace. La traducción resuelve los usos léxicos reales por identidad. Cada procedimiento registra sus referencias globales directas y luego hereda transitivamente las dependencias de sus llamados, uniendo repetidamente los conjuntos de dependencias hasta que ninguno crece. Este punto fijo monótono y finito también cubre grafos de llamadas cíclicos, incluidas las dependencias alcanzadas mediante aristas salientes de un ciclo. Cada llamada en main registra los enlaces globales inicializados disponibles en su propio sitio de llamada. Tras resolver el grafo, un enlace faltante rechaza la compilación con `call requires initialized global binding s<scope>.<name>` en esa llamada. Las lecturas **y escrituras**, incluidas las referencias omitidas estáticamente, requieren inicialización de forma conservadora; no se inventa ningún valor cero o por defecto.

La declaración de un llamador no necesita ver las globales transitivas de un llamado: un llamado hacia adelante puede declararse después de que esas globales se inicialicen. Por ejemplo, una función `first` declarada antes de la global `g` puede llamar a una función posterior `later` cuyo cuerpo lee `g`; llamar a `first` después de que `g` se inicializa es seguro. Llamarla antes de ese inicializador se rechaza. Una lectura directa de la `g` posterior dentro de `first` sigue sujeta a la regla semántica existente de visibilidad por orden de declaración.

Metadatos de procedimientos e instrucciones

`Program.instructions` sigue siendo la cinta ejecutable principal. Las declaraciones de función no se ejecutan en línea. El campo opcional `procedures` se agrega después de los campos heredados de Program y se omite por completo en programas sin funciones. Cada procedimiento contiene, en orden: `label`, `scopeId`, `span`, `parameters` ordenados, `resultKind`, `frame`, `instructions`, `peakTemporaries` y, opcionalmente, las `globals` transitivas ordenadas. Las etiquetas usan la identidad global del símbolo (`s1.increment`); las etiquetas de bifurcación son locales a su cinta. El span del procedimiento es el span de la declaración, y todas las operaciones conservan los spans de la fuente que las originaron.

Los campos de instrucción conservan su orden y etiquetas anteriores, seguidos de los opcionales `callee`, `arguments` y `returnValue`. Los argumentos de llamada son operandos del marco del llamador en el orden de los parámetros formales; una llamada escalar tiene un `target` temporal tipado, y una llamada con resultado null tiene un target no utilizado con valor cero. Un retorno usa `returnValue` cuando es escalar; los campos heredados target/left no utilizados tienen valor cero y no son lecturas ni escrituras. Las listas de argumentos vacías y los valores de retorno ausentes se omiten en JSON.

Todos los valores de los argumentos deben leerse/copiarse **antes** de construir el marco del llamado o entrar en él, incluso cuando el llamador y el llamado usan el mismo nombre `t0`. Las celdas vivas del llamador permanecen reservadas durante las llamadas. Al retornar, se lee el resultado antes de descartar al llamado, se restauran el llamador y la continuación, y se escribe el destino del llamador. Cada invocación tiene almacenamiento nuevo para parámetros, locales y temporales; la ruta heredada de llamada directa comparte solo el almacén global. Las llamadas pasan los escalares por valor; los programas con closures además retienen celdas léxicas compartidas, como se describe más abajo.

Disposición de celdas y representación textual

`frame.cells` es el número total de celdas. `frame.slots` lista, en este orden exacto, `offset` contiguo comenzando en cero, `size: 1`, `role`, `operand` propio opcional y `dynamic` opcional:

Orden	Rol	Contenido
0	`continuation`	Dirección de retorno / posición de la instrucción suspendida del llamador
1	`controlLink`	Activación que hace la llamada
2	`globalAccessLink`	Solo el almacén global compartido, no un entorno de closure

Orden	Rol	Contenido
siguiente, solo resultado no null	`result`	Valor de resultado declarado, incluidos valores de referencia; ausente para resultados null
siguiente	`parameter`	Operandos formales ordenados con tipos declarados o valores desconocidos etiquetados
siguiente	`local`	Todas las variables/constantes propias de la función, incluidos locales de bloque y de `for`, ordenadas por desplazamiento de la declaración en la fuente, con desempate por identidad del operando
última	`temporary`	`t0` hasta `t<peak-1>`, celdas de tipo dinámico

Todos los tamaños y desplazamientos de celdas son **celdas abstractas**, no bytes, palabras de máquina, registros físicos ni una ABI destino completa. Los nombres de operandos de parámetros y locales codifican la pertenencia al ámbito semántico. El operando del slot de un temporal tiene un tipo vacío y `dynamic: true`: sus ocurrencias individuales en instrucciones llevan sus tipos, porque el pool existente puede reutilizar una celda primero como entero y luego como booleano o flotante. No se introduce un segundo asignador. Main no adquiere metadatos ficticios de marco de procedimiento.

Cuando hay procedimientos, el texto comienza con `main:` y luego imprime las instrucciones de main, seguidas de `procedure <label>(<formals>) -> <result> [scope=<id> frame=<n> cells]:`, una línea `cell <offset> <role> [operand] [dynamic]` por slot, las instrucciones del procedimiento y `end <label>`. Sin procedimientos, el texto y el JSON anteriores (incluidos el orden de campos, la numeración de temporales y la ausencia de disposiciones) no cambian.

`functions.cps` termina sin imprimir y calcularía `total = 7` y `result = 8.0`. La CLI emite su TAC, no la salida de su ejecución. La ejecución acotada de marcos, solo para pruebas, verifica el almacenamiento independiente por invocación, la restauración de continuación y control, los efectos globales, los slots definidos, la pertenencia y los límites de etiquetas, pasos y profundidad.

Closures y celdas léxicas retenidas

`closures.cps` instala un contador en un alias de función global inferido. Después de que `install(10)` retorna, llamar a la función guardada calcula `first = 11` y `second = 12`. Copiar ese alias preserva el mismo contador. Llamar de nuevo a `install` crea un contador distinto; los closures hermanos de una misma instalación comparten una celda. La CLI solo genera código; la prueba ejecutable usa el oráculo de prueba acotado.

Contrato explícito del IR

Operación/metadatos	Contrato
`Program.environments`	Disposiciones opcionales ordenadas por `scopeId` semántico, con `parentId` y `bindings` tipados ordenados por identidad. Una disposición no es una asignación viva.
`enterEnvironment`, `environment`	Asigna celdas indefinidas nuevas cada vez que esta instrucción se ejecuta. Enlaza la instancia padre léxica realmente activa en ese momento. Los parámetros de función entran inicializados a partir de las celdas existentes del marco de argumentos; las celdas globales usan el almacén global compartido.
`makeClosure`, `callee`, `environment`, `target`	Construye un valor `function` que empareja la identidad del procedimiento con la instancia exacta del entorno de definición retenido. No lee las capturas ni las inicializa.
`callIndirect`, `left`, `arguments`, `signature`, `target`	Lee el invocable seleccionado y los argumentos, verifica su firma esperada completa (incluidos contratos nominales y de lista) y los requisitos de inicialización, y luego invoca usando su padre léxico retenido. Las llamadas a valores desconocidos protegidas usan el contrato real del invocable, como se describe en T10.

Operación/metadatos	Contrato
`Procedure.lexicalScope`	Disposición del entorno de definición que requiere el procedimiento. Su propio ámbito de función se crea en cada invocación.
`Procedure.captures`	Requisitos transitivos ordenados de enlaces no globales, incluidas lecturas y escrituras. Los requisitos propiedad de un llamador se verifican en su sitio de llamada en vez de propagarse fuera de esa activación.

El texto incluye las declaraciones de disposiciones de entorno y las instrucciones explícitas de entrada a entornos, construcción de closures y llamadas indirectas. Los campos JSON opcionales se omiten en programas que usan solo funciones directas soportadas previamente. El texto/JSON existente de llamadas directas, los desplazamientos de marco y la numeración de temporales no cambian.

En un programa con closures, las celdas de entorno son el almacenamiento autoritativo de los símbolos léxicos; los slots de parámetros y locales del marco siguen siendo el inventario de pertenencia, con los argumentos preparados en los slots de parámetros antes de la entrada. Los temporales, resultados y celdas de control conservan el asignador y el comportamiento de marco existentes. Este es un contrato de almacenamiento abstracto, no una ABI destino ni un recolector de basura. La retención debe sobrevivir al retorno de quien define; volver a entrar en un bloque, incluido el cuerpo de un ciclo, asigna una nueva instancia de entorno. Las capturas de un abuelo siguen los enlaces padre retenidos. No se necesita un opcode de salida de bloque: los operandos y la construcción de closures nombran identidades estáticas de disposición, y las nuevas entradas reemplazan solo la instancia activa, nunca las instancias ya retenidas por closures.

Seguridad de inicialización y de firmas

Las declaraciones de función se elevan (hoisting); la inicialización de variables no. Se permite crear un alias de un closure elevado antes de que su variable capturada se inicialice; llamarlo requiere celdas inicializadas. Las aristas de llamada directa y anidada conocidas usan los requisitos globales de punto fijo existentes más los requisitos de captura léxica. Una llamada anidada insegura conocida estáticamente se rechaza de forma atómica con `call requires initialized captured binding s<scope>.<name>`. Los cuerpos anidados no llamados e inalcanzables se validan igualmente por completo.

Los alias pueden reasignarse dinámicamente. Actualmente, Analyze compara clases de función, no firmas de función completas, en la asignación; esta unidad deliberadamente no cambia esa API semántica. Por ello, una llamada indirecta tipada lleva la `signature` esperada estáticamente; un backend debe rechazar clases de parámetros reales, aridad o clase de resultado que no coincidan **antes de invocar**. El oráculo de prueba reporta `indirect call signature mismatch`. También debe verificar las `globals` y `captures` transitivas del procedimiento real contra su entorno retenido antes de invocar, incluidos los requisitos de solo escritura. Las celdas indefinidas producen un fallo en vez de inventar un valor por defecto. Así, una llamada temprana seleccionada dinámicamente a una global o captura puede compilar pero fallar en la ejecución; no es una nueva garantía estática de inicialización definitiva. Una llamada indirecta alcanzada a través de otra llamada directa o recursiva mantiene la misma guarda. Los diagnósticos estáticos siguen descartando todo el TAC, incluidos los inicializadores de alias inválidos y las guardas estáticas de ámbito/inicialización fallidas en otras partes del grafo de funciones.

El llamado se selecciona y se congela antes de evaluar los argumentos. Los argumentos y los operandos escalares anteriores permanecen congelados a través de llamadas con efectos, usando el pool de temporales existente. Copiar una función copia su par procedimiento/entorno, no instantáneas de los escalares capturados. La aritmética y las comparaciones conservan verificaciones de tipo explícitas, con operaciones protegidas sobre valores desconocidos descritas más abajo. Los resultados de función siguen su tipo declarado, incluidos los contratos de referencia, y los resultados omitidos siguen siendo `null`; no hay anotaciones de retorno de tipo función, retornos de closure inferidos ni nuevas lambdas.

Condiciones y flujo de control

Las comparaciones se evalúan de izquierda a derecha y producen booleanos; los operandos numéricos mixtos reciben conversiones explícitas de entero a flotante. La aritmética numérica requiere operandos numéricos, no booleanos; la suma de cadenas tiene su propio contrato. Se soportan literales booleanos, declaraciones y constantes inicializadas, asignaciones a identificadores, sentencias de expresión y el `!` unario.

`a && b` bifurca hacia `b` solo cuando `a` es verdadero; `a || b` bifurca hacia `b` solo cuando `a` es falso. El agrupamiento y la negación anidada conservan esta semántica. Una expresión lógica usada como valor reserva un temporal de resultado vivo antes de traducir sus operandos, y luego escribe true o false en la arista entrante correspondiente

antes de la unión. Las operaciones recursivas del lado derecho no pueden reciclar ese resultado reservado. Una condición puede, en cambio, bifurcar directamente sin materializar un valor lógico.

`if/else` traduce ambos cuerpos pero ejecuta solo el cuerpo seleccionado. Los `if` anidados, `else if`, sin `else`, vacíos y sin llaves usan los envoltorios de bloque existentes del frontend y los ámbitos semánticos hijos correspondientes. Cada inicializador de declaración lee los enlaces externos antes de activar su propio nombre; los nombres locales a una rama no escapan. Ambos lados de una expresión lógica y ambas ramas se validan siempre de forma estática, incluso tras una condición literal. La sintaxis no soportada o los errores semánticos en fuente inalcanzable siguen rechazando la compilación completa.

El ejemplo condicional protege la división entre cero y selecciona `quotient = -1`. La CLI emite solo TAC; las aserciones de ejecución usan un intérprete acotado solo para pruebas, no un runtime ni un optimizador de producto.

Límite del compilador de fuente

Ciclos y transferencias

Construcción	Orden de ejecución	Destino de `continue` del ciclo más cercano
`while`	Condición, cuerpo, repetir	Condición
`do-while`	Cuerpo, condición, repetir si es verdadera	Condición
`for`	Inicializador una vez, condición, cuerpo, post, repetir	Post, luego condición

`break` sale del ciclo o switch más cercano, omitiendo el post de un `for` al salir de ese ciclo. Una condición de `for` omitida es verdadera; el inicializador y el post son opcionales de forma independiente. Los ciclos anidados y consecutivos conservan destinos de transferencia separados. Todos los cuerpos y posts se validan estáticamente incluso cuando una condición literal impide su ejecución; el rechazo no emite TAC.

Un inicializador de `for` usa el ámbito semántico de ciclo existente, con un ámbito hijo para el cuerpo. Los cuerpos de while/do usan sus ámbitos de bloque; la condición de un do no puede ver los locales del cuerpo. Los inicializadores que sombrean leen los enlaces externos antes de activar sus nombres. Los temporales del encabezado, cuerpo y post se consumen antes de las aristas de retorno (backedges); las iteraciones en tiempo de ejecución reutilizan las identidades de temporales emitidas sin aumentar la demanda estática de asignación.

Un post de `for` completo puede ser una expresión de asignación simple a un identificador, opcionalmente agrupada: `for (;; (i=i+1)) { if(i>=3) break; }`. Su ruta de almacenamiento previa y su IR no cambian. T9 además soporta asignaciones a identificadores, propiedades e índices que producen valor en las posiciones de expresión aceptadas por la gramática existente.

El frontend ahora identifica la condición y el post por sus delimitadores reales del encabezado, no por la cantidad de expresiones. Los inicializadores de declaración y de asignación consumen su propio primer punto y coma. La excepción de compatibilidad intencional son los encabezados con condición omitida y post presente, para toda variante de inicializador: `for (;; i=i+1) { ... }` ahora no tiene condición y tiene un post de asignación, en vez de una condición de asignación que reportaba espuriamente `SEM_CONDITION` / `condition must be boole an`. Esto también corrige los informes públicos de AST y de análisis para esa familia. La gramática, el esquema del AST y las políticas semánticas no cambian; los demás informes de análisis existentes se preservan.

El ejemplo de ciclo, que termina, calcula `total = 13` sin imprimir ni usar un runtime de producto. Ejercita while-continue, do-while y break/continue en `for`.

Switches booleanos

El selector debe ser booleano (se verifica en tiempo de ejecución para valores desconocidos); los casos conservan las reglas del analizador de tipo compatible y de casos estáticamente únicos. Los selectores numéricos siguen siendo errores semánticos, como lo exige la política del equipo en `../entrega1/compliance.md`. El selector se evalúa una sola vez, incluso en un switch vacío o solo con default. Las expresiones de caso se ejecutan en orden de la fuente hasta la primera coincidencia; default se ejecuta solo si ninguna coincide. Cada cuerpo seleccionado sale implícitamente, incluido un caso vacío: no hay caída (fallthrough).

El despacho precede a todas las etiquetas de los cuerpos. Un temporal del selector permanece vivo y sin cambios durante todas las comparaciones; cada comparación usa un slot de resultado booleano separado. Todas las pruebas y cuerpos se traducen y validan, incluida la fuente no seleccionada. Cualquier error descarta todo el

programa TAC. El despacho del switch en sí usa los opcodes existentes; sus expresiones de caso ahora pueden contener llamadas directas escalares.

Los marcos de switch y de ciclo comparten una pila de transferencias. `break` sale de su marco más cercano; `continue` salta los marcos de switch hasta la etiqueta de condición o de post de `for` del ciclo más cercano. Un switch por sí solo no hace legal un `continue`.

Las sentencias de caso comparten el ámbito **semántico** entrante y los IDs de símbolo; solo los bloques reales crean ámbitos hijos. El TAC exige de forma conservadora que los enlaces compartidos estén inicializados antes del switch. Un caso puede inicializar y usar su propia declaración local, pero los casos posteriores y el código después del switch no pueden leer ni asignar ese enlace exclusivo del caso. Esta restricción también se aplica a switches solo con default y a declaraciones dentro de switches anidados que comparten el mismo ámbito. Tales usos reportan `TAC_UNSUPPORTED` en la fase `codegen` con el mensaje ubicado `switch binding is not guaranteed initialized on this path`.

Esta es una restricción de capacidad de la generación de código, no un cambio en el ámbito del lenguaje ni en los informes de análisis. Los nombres bloqueados no pueden recurrir silenciosamente a un enlace externo sombreado por la declaración de un caso anterior. El propio inicializador de una declaración sigue leyendo el nombre externo antes de la activación. Los enlaces externos inicializados siguen siendo legibles y escribibles a través de los casos y bloques anidados; no se inventan valores por defecto.

El ejemplo de switch calcula `total = 6` sin imprimir. Demuestra el despacho ordenado, la salida implícita, el break de switch y el continue a través de un switch hasta el post de un `for`.

Objetos, métodos enlazados y cadenas

Genere el ejemplo de objetos sin impresión con cualquiera de los formatos de la CLI:

```
go run ./cmd/compiscript-tac testdata/compiscript/tac/objects.cps
go run ./cmd/compiscript-tac -format json testdata/compiscript/tac/objects.cps
```

El oráculo de prueba acotado ejecuta el JSON emitido hasta `observed = 9`, `first = 11`, `second = 12`, `independent = 21`, `shared = 30` y `label = "counter:ready"`. La CLI genera TAC; no es un intérprete.

Asignación de memoria e inicialización

Cada asignación tiene identidad de referencia y sus propias celdas de campo, inicialmente indefinidas. Las copias, los parámetros y resultados de clase, los almacenamientos en campos y las asignaciones a subtipos conservan esa identidad; no copian los campos de la instancia. La igualdad compara la identidad del objeto. Una referencia de clase null puede almacenarse y compararse, pero el acceso a miembros produce el fallo `null receiver` en el oráculo.

La construcción evalúa los argumentos una vez, de izquierda a derecha, y toma instantáneas de ellos a través de las llamadas a inicializadores. Luego **todos los inicializadores de campos heredados y propios se ejecutan de la base a la derivada, en orden de declaración dentro de cada clase**. Solo después se ejecuta una vez el constructor más cercano seleccionado. Una subclase sin constructor propio usa el constructor heredado más cercano; una subclase con constructor propio no llama automáticamente a ningún constructor ancestro. Este orden es la decisión explícita del usuario en S3, no un comportamiento por defecto tomado de JavaScript o TypeScript.

Un campo sin inicializador permanece indefinido hasta que se escribe. Leerlo produce el fallo `undefined field`; no se fabrica un valor por defecto cero, cadena vacía ni null. Los inicializadores de campos constantes pueden escribir su propio campo exactamente una vez; la mutabilidad ordinaria y el acceso a miembros siguen gobernados por Analyze. Los inicializadores y métodos se validan incluso cuando su clase no se usa. El código no soportado rechaza la compilación completa.

IR explícito de objetos

Operación/metadatos	Contrato
`Program.classes`	Disposiciones de clase opcionales en orden de ámbito semántico. Cada declaración tiene un `id` calificado por ámbito, nombre, identidad del padre, ámbito léxico, span de la fuente, disposición de campos, tabla de métodos virtuales y procedimiento inicializador.

Operación/metadatos	Contrato
Disposición de campos	Los campos heredados conservan su identidad de declaración, propietario y desplazamiento. Los campos propios se agregan al final en orden de declaración. Las celdas llevan tipos semánticos completos, identidades de clase nominales opcionales, mutabilidad y spans de la fuente. Los desplazamientos cuentan celdas abstractas, no bytes.
`newObject`, `class`, `target`	Asigna celdas de campo indefinidas nuevas y retiene los entornos léxicos de definición reales de la jerarquía de clases. La identidad de la asignación es distinta de la identidad estática de la disposición.
`loadField`, `left`, `class`, `member`, `offset`, `target`	Lee un campo del receptor seleccionado después de validar la pertenencia nominal y la identidad de la disposición.
`storeField`, `left`, `right`, `class`, `member`, `offset`	Lee el receptor y el valor antes de la mutación. El receptor se retiene a través de las llamadas del lado derecho; ninguna expresión receptora se vuelve a evaluar.
`bindMethod`, `left`, `class`, `member`, `target`	Selecciona la implementación a partir de la clase real del receptor y la empareja con ese receptor y su entorno de definición. La selección ocurre antes de evaluar los argumentos.
`bindMethod`, `callee`	La selección interna de inicializadores nombra el procedimiento inicializador exacto en vez de usar despacho virtual. La construcción emite una llamada a inicializador por clase y luego, como máximo, una llamada a constructor.
`Procedure.receiver`	Celda receptora implícita opcional, separada de los parámetros explícitos de la fuente. Entra en la misma infraestructura de marco/entorno que los parámetros. Las funciones anidadas capturan esa celda léxica, no el receptor actual de un llamador.
Operando `class`, `Procedure.resultClass`, `typeClasses` de llamada	Identidades nominales opcionales calificadas por ámbito para valores de clase y verificaciones completas de firma en llamadas indirectas. Una llamada indirecta verifica las identidades de parámetros y resultado, además de clases y aridad, antes de la invocación.

Los métodos enlazados usan los `callIndirect`, marcos, resultados, guardas de inicialización, entornos retenidos y asignación de temporales existentes. Las funciones anidadas que escapan conservan su `this` léxico; los alias de métodos enlazados conservan el receptor seleccionado cuando se creó el alias. Los nombres de campos y métodos no calificados usan el ámbito léxico de clase del analizador: el acceso a propiedades heredadas sigue disponible mediante `this`, pero un campo heredado no se inserta como un nuevo enlace léxico no calificado.

Las cadenas soportan literales, almacenamiento, parámetros y resultados tipados, concatenación e igualdad/desigualdad. La compatibilidad de almacenamiento de cadenas y clases está separada de la aritmética numérica; las referencias no se aceptan como operandos numéricos arbitrarios. Los metadatos de clase nominales se omiten cuando no se usan, preservando el texto/JSON del TAC soportado previamente. El oráculo rechaza identidades de clase malformadas, ciclos de herencia, desplazamientos y propietarios de campos, inicializadores o métodos faltantes y llamadas dinámicas incompatibles. Estas guardas documentan un contrato abstracto del backend, no nuevos diagnósticos estáticos.

Los alias inferidos conservan la identidad de clase calificada por ámbito del inicializador incluso cuando una clase anidada sombrea su nombre de tipo textual. La misma identidad se registra en el almacenamiento de entorno y de marco; las lecturas de campos y los resultados de funciones conservan sus orígenes de declaración. Las anotaciones explícitas y los almacenamientos tipados existentes siguen imponiendo sus identidades nominales declaradas. Esto usa metadatos privados de traducción en vez de cambiar Analyze o introducir otro modelo de enlace en el frontend.

Listas, indexación y foreach

Las listas son referencias de longitud fija, no arreglos copiados. La asignación, los argumentos, los resultados, los campos y los elementos anidados conservan la identidad de referencia; `==`/`!=` comparan identidades. Cada ejecución de un literal asigna una referencia distinta, incluso para `[]`. Los literales vacíos contextuales, anidados

e irregulares usan los tipos de elemento semánticos existentes. La validación de aritmética escalar no cambia. No existe un miembro `.length` en la fuente ni una API para agregar elementos.

Instrucción	Operandos	Contrato
`newList`	`target`, `arguments` ordenados	Lee los valores de los elementos y luego asigna una referencia nueva
`listLength`	`target`, `left`	Lee la longitud interna de la referencia en un entero
`loadIndex`	`target`, `left`, `index`	Verifica los límites del índice entero y lee un elemento
`storeIndex`	`left`, `index`, `right`	Verifica límites y tipo, y reemplaza un elemento sin redimensionar

El campo `shape` del operando registra la cadena completa de clases de elemento separadas por barras: un operando `integer[][]` tiene `type: "list"`, `shape: "list/integer"`. Las hojas nominales también conservan su identidad `class` calificada por la declaración, incluso bajo sombreado. Los campos y los resultados de procedimientos llevan metadatos opcionales `shape`/`resultShape` correspondientes; las firmas indirectas verifican la forma anidada completa y los orígenes nominales antes de entrar en un llamado. Estos campos se omiten en programas que no usan listas. Los elementos de lista con valor de función también conservan sus identidades nominales originales de parámetros y resultado a través de la indexación anidada, los alias inferidos y los enlaces de foreach. Extraer una función nunca vuelve a resolver los nombres de clase de su firma en el ámbito de la extracción; la guarda existente de llamada indirecta sigue rechazando reemplazos incompatibles antes de ejecutar el cuerpo de la función seleccionada.

Los elementos literales y las expresiones de colección, índice y lado derecho se evalúan una vez, de izquierda a derecha. Los operandos de lista e índice se congelan antes de que efectos posteriores puedan reasignar sus enlaces fuente. Las lecturas preceden a las escrituras del destino; los valores vivos usan el pool de temporales y los marcos de invocación existentes, incluidos los llamadores recursivos. Un almacenamiento verifica los límites de su destino después de evaluar su lado derecho, de modo que un efecto del lado derecho puede preceder a un fallo de límites en tiempo de ejecución.

Foreach evalúa su iterable una vez y retiene esa referencia inicial y su longitud interna. Esta es la política práctica de longitud fija: ninguna operación de la fuente puede redimensionar una lista. Reasignar el enlace fuente no redirige la iteración; las mutaciones de la lista retenida son visibles para las entradas siguientes. Cada entrada real al cuerpo entra en su ámbito semántico de bloque existente, inicializa un enlace de iteración mutable nuevo y luego ejecuta el cuerpo. Asignar ese enlace no reemplaza el elemento en sí; mutar una referencia de objeto o lista a través de él sí afecta a los alias. Los closures que escapan retienen celdas de entrada distintas. Continue avanza el índice privado; break sale; los ciclos y switches anidados reutilizan la pila existente de transferencia más cercana.

Los diagnósticos semánticos de raíz constante, de tipo y de límites estáticos siguen siendo la autoridad y descartan todo el informe TAC. Los fallos dinámicos de null, límites y tipo son obligaciones del backend del IR, ejercitadas solo por el oráculo de prueba acotado, no nuevos diagnósticos de Analyze. El analizador permite vistas covariantes de listas (incluido el ampliación numérico); el oráculo ensancha las lecturas de elementos enteros a través de una vista flotante y protege con guardas las lecturas posteriores incompatibles tras la mutación de un alias. Nunca reinterpreta silenciosamente un objeto nominal o una forma anidada incompatible. Las listas no son un motor de ejecución de producto ni una ABI destino.

`collections.cps` ejercita listas anidadas compartidas y celdas de iteración capturadas. Su JSON emitido por la CLI se ejecuta en las pruebas hasta `shared = 9`, `sum = 12`, `first = 3`, `second = 6`, `continued = 4`; la CLI en sí emite código, no esos valores.

Fuente soportada

La fuente implementada incluye print, `%`, ternarios perezosos, asignaciones que producen valor, declaraciones no inicializadas con y sin tipo, y valores null. Los valores desconocidos usan instrucciones explícitas protegidas en vez de tipos estáticos fabricados. Los AST malformados, las identidades de ámbito faltantes y los diagnósticos reales se rechazan de forma atómica. Ningún TAC parcial puede escapar después de errores léxicos, sintácticos, semánticos o de traducción. Los bytes originales de la fuente se preservan; Compile conserva los diagnósticos y ámbitos correspondientes de Analyze. Se soportan las sentencias de expresión numéricas; su temporal final se libera sin almacenamiento. Null puede ocupar celdas de enlace, de campo y de elemento de lista, así como almacenamiento de referencia tipado. Los campos sin anotación conservan el contrato desconocido del analizador en sus disposiciones privadas y en cada copia heredada. Un inicializador, incluido null o una llamada

a un alias externo, proporciona un valor sin estrechar el tipo de almacenamiento del campo. La ausencia de inicializadores deja los campos indefinidos; solo las anotaciones explícitas restringen sus etiquetas reales. Los inicializadores siguen ejecutándose una vez bajo la política de constructores A, y los metadatos semánticos no se reescriben. El almacenamiento desconocido, las firmas sin tipo y try/catch usan el contrato T10 descrito más abajo. Las transferencias sin una construcción envolvente elegible se rechazan. Los nodos de recuperación o las invariantes de traducción violadas no se omiten silenciosamente. El rechazo defensivo de declaraciones conserva el mensaje histórico `only integer, float and boolean declarations are supported`; no se usa para rechazar declaraciones soportadas null, de referencia o desconocidas solo por su clase. Los demás diagnósticos defensivos conservan su redacción numérica histórica por compatibilidad.

Valores estáticos y efectos (T9)

`let x=1; let y=x+(x=2);` calcula `y=3`, `x=2`: los operandos, argumentos, llamados, receptores e índices anteriores conservan sus valores según su posición en la fuente a través de efectos posteriores. Las asignaciones producen su tipo/valor almacenado, incluido el ampliación a flotante. Los ternarios reservan un temporal de unión antes de traducir cualquiera de sus brazos, evalúan la condición una vez y ejecutan solo el brazo seleccionado. Ambos brazos se validan igualmente. La forma esperada de listas anidadas y los orígenes nominales y de firma se propagan a través de las uniones. `branchTrue` requiere un booleano en tiempo de ejecución; no hay veracidad numérica (truthiness). Los tipos de brazo incompatibles o desconocidos usan una celda de unión etiquetada y fronteras tipadas verificadas.

`declare` restablece el enlace a indefinido cada vez que se ejecuta su declaración. Volver a entrar a un marco o entorno nunca hereda un valor previo. Leer una celda indefinida produce un fallo en el oráculo acotado. Una celda indefinida activada satisface las guardas existentes de dependencia y tipo, lo que permite la inicialización mediante una llamada; una lectura real sigue fallando. Los enlaces aún no activados siguen sin estar disponibles. La activación de una declaración es distinta de una escritura por defecto, y la celda reservada real queda inventariada por `codegen`, incluso cuando la fuente nunca la asigna. Las verificaciones existentes de dependencias de llamadas permanecen intactas.

El residuo usa la semántica de `math.Mod` de Go: `x-trunc(x/y)*y`, con el signo del dividendo. Dos enteros producen un residuo entero; los operandos flotantes o mixtos producen un flotante tras un ampliación explícito. Un divisor cero produce un fallo antes de escribir el destino. Esta es la extensión práctica KISS de la promoción numérica existente, no un módulo euclidiano.

Print usa `left` y un span de la fuente, sin destino ni llamada sintética. La CLI genera TAC; no ejecuta print. El oráculo solo para pruebas comparte un único colector de salida entre marcos y agrega un salto de línea por cada print. Las cadenas se imprimen tal cual; los booleanos son `true`/`false`; null es `null`; los números usan el texto binary64 más corto que permita ida y vuelta exacta. Las referencias usan `<object s1.A>` / `<function s1.f>`, nunca direcciones físicas. Las listas usan `[1, 2]`, con detección de ciclos activos, un presupuesto de 64 valores y un límite de profundidad de 8 (`<cycle>` / `...`). Esta representación determinista y acotada evita un nuevo runtime o una dependencia de serialización; no es un intérprete de producto.

Valores protegidos y excepciones síncronas (T10)

La clase semántica `error` es un marcador de información ausente, **no** un nuevo tipo `any` en la fuente ni una afirmación de que las ejecuciones inválidas están bien tipadas. Un operando `error` almacena la etiqueta/referencia real en tiempo de ejecución. Los parámetros formales sin tipo, los alias, las capturas, los resultados de miembros e índices y las uniones la conservan. Los programas existentes con tipos estáticos no necesitan metadatos nuevos: `guarded` se omite salvo que se use.

Instrucción / metadatos	Contrato
Instrucción aritmética/de comparación/unaria con `guarded:true`	Verifica los operandos reales; promoción numérica, comparaciones numéricas y suma de cadenas cuando son válidas; negación booleana estricta; los operandos inválidos producen un fallo
`checkValue target = left`	Valida el valor real antes de una frontera tipada; se permite el ampliación de entero a flotante; los orígenes nominales y los contratos de rango y hoja de listas siguen verificándose
`callIndirect` con `guarded:true` y sin `signature`	Selecciona el invocable real una vez; verifica su contrato formal real completo antes de los efectos del cuerpo; usa su contrato de resultado real

Instrucción / metadatos	Contrato
`dynamicMember` / `dynamicStore`	Resuelve campos y métodos enlazados en el objeto real; preserva propietario, mutabilidad y tipo del valor almacenado; los receptores faltantes, null o que no son objetos producen un fallo
`pushHandler label` / `popHandler`	Registra/elimina un manejador en la invocación dinámica actual, no en un entorno léxico retenido
`catchException target`	Consume la referencia opaca `exception` pendiente en un ámbito de catch nuevo; el manejador seleccionado ya fue eliminado

Las operaciones existentes `copy`, de lista y de bifurcación conservan sus verificaciones explícitas. Las celdas desconocidas no tienen un valor de inicialización por defecto inventado. Las condiciones nunca usan veracidad (truthiness). Las referencias a listas vacías o genéricas conservan su identidad de asignación; un uso tipado verifica una vista o su contenido sin copiar el contenedor ni cambiar silenciosamente sus restricciones. Los contratos de lista tipados incompatibles conocidos siguen siendo estrictos. La mutación de vistas covariantes tipadas existente y el momento de sus fallos no cambian.

Las anotaciones de resultado de función faltantes siguen significando **null**, no resultados inferidos. Los retornos desconocidos no null de estas funciones producen un fallo antes de la limpieza de manejadores. Null sigue siendo null a través de vistas de referencia tipadas y parámetros formales desconocidos. La igualdad protegida permite la promoción numérica y las comparaciones de identidad null/referencia; las etiquetas en tiempo de ejecución no null incompatibles producen un fallo en vez de inventar coerciones. Las llamadas dinámicas con resultado null se ejecutan una vez y escriben null en el destino etiquetado del llamador sin agregar una celda de resultado al llamado. La declaración real del invocable, el receptor, el entorno léxico y la firma nominal completa permanecen adjuntos a la referencia a través del almacenamiento desconocido y de las colecciones.

Política KISS de excepciones: los fallos explícitos del lenguaje (acceso indefinido o null, límites, división/residuo entre cero, tipo/llamado/miembro/índice/condición dinámicos inválidos y guardas nominales/de firma en tiempo de ejecución) son capturables. Sus mensajes se mantienen estables; el oráculo acotado imprime el mensaje de una excepción sin direcciones físicas ni trazas de pila internas. Cada excepción entregada tiene identidad de referencia opaca. Los spans de la fuente de las instrucciones y del enlace de catch permanecen en el IR; los mensajes no inventan ubicaciones de pila del anfitrión. Los efectos anteriores al fallo quedan confirmados, no se revierten.

Gana el manejador más cercano en la **pila dinámica de llamadas**, abandonando las continuaciones de los llamados intermedios. Las celdas de closures que escaparon de llamadas abandonadas permanecen retenidas. Los fallos dentro de un catch llegan a un manejador externo porque el manejador seleccionado se elimina antes de entrar al cuerpo del catch. Una ruta normal de try omite el catch. Retornar valida y lee su valor **antes** de la limpieza; return/break/continue que escapan eliminan solo los manejadores cruzados, mientras que las transferencias que permanecen dentro de un try conservan su manejador. Las nuevas llamadas no heredan manejadores del entorno de definición de su closure.

El IR malformado, los fallos por presupuesto de recursos y los errores del anfitrión no son excepciones del lenguaje. El oráculo verifica de antemano los contratos de instrucciones, marcos y disposiciones y usa tipos de fallo explícitos, nunca clasificación por subcadenas de mensajes ni recuperación generalizada de panics. Sigue siendo solo para pruebas: 10,000 instrucciones, 64 niveles de llamada y tamaños de marco acotados. La CLI y la API generan TAC; no ejecutan este oráculo.

Cada clase válida de nodo del AST tiene un despacho de traducción: las sentencias Block, VarDecl, ConstDecl, Assign, FunctionDecl, ClassDecl, Expr, Print, If, While, DoWhile, For, Foreach, TryCatch, Switch, Break, Continue y Return; las expresiones Identifier, Literal, Array, This, New, Group, Unary, Binary, Ternary, Assign, PropertyAssign, Call, Index y PropertyAccess. `BadStmt`/`BadExpr` son nodos de recuperación y se rechazan. Este inventario de clases no garantiza que toda ejecución aceptada tenga éxito: los fallos dinámicos y las guardas conservadoras de inicialización siguen siendo intencionales.

Ruta rápida de la biblioteca

Dentro de este módulo de Go, llame a `compiscript.Compile(source)` desde `genanalex/internal/compiscript`. El resultado es un `CompilationReport` con los campos JSON `tac`, `diagnostics`, `scopes` y, en caso de éxito, el opcional `codegen`. El análisis sintáctico y el semántico se ejecutan una vez cada uno. Cualquier error del frontend o semántico preserva los mismos diagnósticos ordenados e instantáneas de ámbitos que `Analyze`, fija `tac` en `null` y omite por completo la traducción. La traducción reporta una guarda estática de inicialización/ámbito

fallida o una invariante violada con el código `TAC_UNSUPPORTED`, la fase `codegen`, un mensaje descriptivo y su span de la fuente; `tac` también es `null`, incluso cuando las sentencias anteriores estaban soportadas. Una fuente vacía tiene éxito con `instructions: []`, `peakTemporaries: 0` y `diagnostics: []`.

Por ejemplo, `let x=1/2; x=x+3;` produce exactamente:

```
t0:float = i2f 1:integer
t1:float = i2f 2:integer
t0:float = t0:float / t1:float
s1.x:float = t0:float
t0:float = i2f 3:integer
t0:float = s1.x:float + t0:float
s1.x:float = t0:float
```

El máximo es de dos temporales: la división consume `t0` y `t1` antes de reutilizar `t0` como su destino. Cada instrucción JSON también conserva su span de la fuente y las clases de sus operandos. El compilador no evalúa ninguna aritmética.

Para `let x=1; print(x);`, el resultado incluye `print s1.x:integer`. Para `let x:integer;`, emite `declare s1.x:integer` sin un valor fabricado. Para `let x=missing;`, se devuelve el diagnóstico semántico existente en vez de intentar la generación de código. El comando por defecto representa el mismo TAC; el JSON expone el informe completo.

Límites restantes

- Las guardas conservadoras de inicialización para enlaces de switch, globales y capturados siguen siendo intencionales; esta entrega no las reemplaza con un análisis de rutas más fuerte.
- Tamaños y desplazamientos en bytes de la máquina destino y ABI de llamadas, recolección de basura y enlaces léxicos específicos de la máquina destino. Se implementan entornos léxicos retenidos abstractos y celdas de función de parámetros, locales, temporales, resultado y control, no esta disposición destino.
- La brecha original de construcción queda resuelta por T9/T10; el informe final no establece ninguna nueva brecha concreta de implementación normativa. Los controles humanos y de evaluación permanecen en la lista de verificación del informe.
- La salida del IDE de fuente a TAC, las disposiciones estáticas de main y globales, y los ejemplos mixtos exitosos y fallidos están implementados; la evaluación visual en un navegador real sigue pendiente.

El acceso a memoria futuro necesita opcodes distintos y operandos explícitos, como ya los tienen las llamadas directas; las operaciones no deben colarse dentro de cadenas de símbolos numéricos. Las referencias a objetos y colecciones usan el contrato explícito de valor/almacenamiento; no se codifican como cadenas de operandos escalares. El panic por cuerpo de ciclo faltante se corrigió con una búsqueda de bloque protegida; la batería de aceptación verifica el rechazo atómico y ubicado sin recuperación generalizada. Esto no es una prueba para toda entrada malformada.

Verificación

```
GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local go test ./internal/compiscript ./internal/compiscript/tac -co
GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local go vet ./internal/compiscript ./internal/compiscript/tac
GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local go test ./cmd/compiscript-tac -count=1
```

Las pruebas actuales fijan el formato de las instrucciones, la salida determinista del programa, la unicidad de los temporales vivos, la protección de la liberación, la reutilización con cambio de tipo y la asignación acotada a través de expresiones repetidas. Las pruebas a nivel de fuente fijan precedencia, asociatividad, ampliación, división, enlace léxico, instrucciones exactas tipadas y ubicadas en la fuente, rechazo sin salida parcial, JSON determinista, preservación de la entrada y compatibilidad con los diagnósticos de análisis. Las pruebas de la CLI ejecutan el binario construido para verificar la salida exacta de TAC y del informe, ejecuciones repetidas, rechazo ubicado, códigos de salida, errores de argumentos y de entrada, y archivos sin cambios. El inventario de construcciones y los casos que pasan no constituyen corrección exhaustiva ni cobertura de líneas medida. Las pruebas de condiciones ejecutan las instrucciones generadas con un presupuesto finito de pasos, validan etiquetas, destinos y temporales definidos, y verifican trampas (traps) de división omitidas y requeridas, tablas de verdad, uniones anidadas, ámbitos de rama, preservación de la fuente y asignación determinista y acotada de temporales. Las pruebas de ciclos usan el mismo intérprete acotado para cláusulas opcionales, aristas de retorno, transferencias al más cercano, sombreado de ámbitos, promoción numérica y trampas de cortocircuito. Las pruebas de switch reutilizan ese intérprete para el orden de despacho, el tiempo de vida del selector, las trampas omitidas, la salida implícita, las transferencias anidadas, las restricciones de inicialización en casos y la presión acotada de temporales. Las pruebas de la CLI cubren el éxito en texto/JSON y el rechazo atómico

por cuerpos omitidos. Las pruebas de funciones ejecutan cintas separadas con marcos abstractos nuevos, congelamiento de valores del llamador, parámetros mutables por valor, ampliación, retornos null y escalares, flujo de control anidado y efectos globales transitivos. Fijan disposiciones, JSON y texto deterministas, celdas temporales dinámicas, fuente/AST/ámbitos inmutables y el rechazo atómico por función, grafo e inicialización, incluidos los cuerpos no llamados. El oráculo de condiciones previo no cambia; el oráculo de marcos es solo para pruebas, con límites de instrucciones y profundidad en vez de una política de desbordamiento de un runtime de producto. Las regresiones recursivas ejercitan factorial, múltiples resultados recursivos vivos, llamadas booleanas mutuas, procedimientos con resultado null, dependencias globales cíclicas y rechazo por inicialización. `recursion.cps` calcularía `product = 120` y `sequence = 21`; la CLI emite TAC, no esos valores. Las pruebas de closures además ejecutan contadores que escapan, invocaciones distintas, celdas hermanas compartidas, propietarios recursivos, capturas de abuelos, reentrada a bloques y ciclos, hoisting, inicializadores que sombrean y el orden de selección del llamado. El JSON de la CLI construida se decodifica y se ejecuta a través del mismo oráculo acotado de entornos retenidos. Las disposiciones inválidas, las celdas de entorno indefinidas o fuera de límites, los desajustes de firma y los fallos de inicialización se verifican sin agregar un intérprete de producto.